BTEUP EXAM NOTES | HINGLISH EDITION

Mobile App Development

Unit III: User Interface Design Essentials: Widgets, Layouts, Animation

In notes ke baare me

Part A me har topic simple Hinglish me real-life example ke saath samjhaya hai, taaki beginner bhi zero se samajh sake.

Part B me important exam questions ke poore 5-marks wale answers Hinglish me diye hain.

Last me: quick revision sheet, common mistakes aur self-test.

Exam me answer English me likhna hai to English Edition PDF use karo.

Syllabus covered (Unit III)

Android User Interface Design Essentials: User Interface Screen elements, Designing User Interfaces with Layouts, Drawing and Working with Animation.

Unit Overview aur Contents

Simple idea: User interface (UI) wo sab kuch hai jo user **dekhta aur touch karta hai**: text, buttons, images, lists. Is unit me teen parts hain: (1) **UI elements** (views aur widgets), (2) unhe screen par arrange karne wale **layouts**, aur (3) screen ko graphic aur lively banane ke liye **drawing aur animation**.

Zaroori note: Views vs Jetpack Compose

Tumhare syllabus me classic **View + XML layout** system padhaya jaata hai, isliye notes usi ko follow karte hain. Par **May 2026 me Google ne announce kiya ki Android UI development 'Compose first' hai**: wo saare naye features ke liye **Jetpack Compose** recommend karta hai. View system chalta rahega (sirf critical bug fixes, naye features nahi) aur bahut se existing apps abhi bhi use karte hain. Exam ke liye Views seekho; layouts topic ke end me chhoti Compose comparison table di gayi hai.

Topic	Exam importance	Notes me kahan
UI basics: View, ViewGroup, attributes	High	Part A: Topic 1 | Part B: Q1, Q14
UI elements (widgets)	Very High	Part A: Topic 2 | Part B: Q2, Q3, Q4, Q5
Layouts	Very High (almost sure)	Part A: Topic 3 | Part B: Q6, Q7, Q8, Q9
Drawing: text, bitmaps, shapes, canvas	Very High	Part A: Topic 4 | Part B: Q10
Animation: tween, frame, property	Very High	Part A: Topic 5 | Part B: Q11, Q12, Q13

Contents

Part A: Easy Explanation	Part B: Important Questions aur Answers
Topic 1: UI basics Topic 2: UI screen elements Topic 3: Layouts se UI design karna Topic 4: Drawing (text, bitmaps, shapes, canvas) Topic 5: Animation ke saath kaam	Q1 View, ViewGroup, UI hierarchy (Likely) Q2 TextView and EditText (Very Likely) Q3 Button, CheckBox, RadioButton, Switch (Very Likely) Q4 Spinner and ListView (Very Likely) Q5 ProgressBar, SeekBar, RatingBar, pickers (Likely) Q6 Layouts and their types (Very Likely) Q7 LinearLayout (Very Likely) Q8 RelativeLayout (Very Likely) Q9 Frame, Table, Grid, Constraint layouts (Likely) Q10 Drawing: Canvas, Paint, shapes, bitmaps (Very Likely) Q11 Types of animation (Very Likely) Q12 Tween animation (Very Likely) Q13 Frame animation (Likely) Q14 Differences (Possible)

Is unit ko kaise padhein (time: 1 din)

1) Part A ek baar padh kar samjho. 2) **Har layout ki XML** aur **chaar tween animations** (alpha, scale, translate, rotate) yaad karo: inke programs puchhe jaate hain. 3) Ek widget program (button click, spinner) aur ek animation program practice karo. 4) Quick Revision Sheet se revise karo aur Self-Test karo.

PART A: Easy Explanation (Hinglish)

Topic 1: UI Basics: View and ViewGroup

Real-life example

Ek **photo frame wali deewar** socho. Deewar screen hai. Har photo frame ek **View** hai (button, text, image). Kai frames ko sahi order me rakhne wala **bada board** ek **ViewGroup** (layout) hai.

Definition

View user interface ka basic building block hai; ye screen par kuch draw karta hai aur user events handle karta hai (TextView, Button). **ViewGroup** ek invisible container hai jo doosre views ko rakhta aur arrange karta hai (LinearLayout, RelativeLayout).

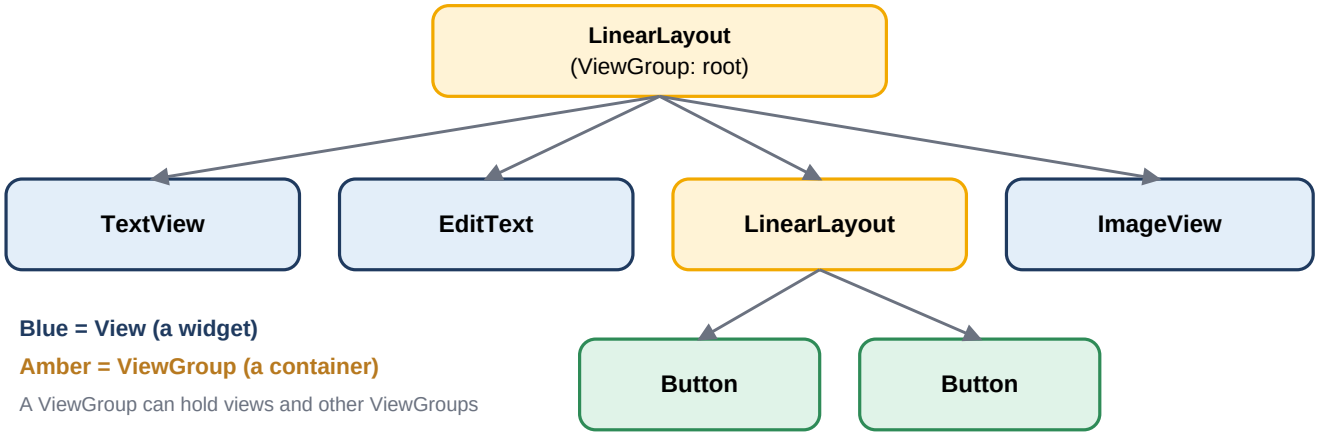


Figure 1: View hierarchy: ViewGroups contain Views

UI banane ke tareeke

- **XML layout files** (res/layout): Views ke liye recommended tareeka. Design code se alag rehta hai.
- Android Studio ka **Layout Editor**: views drag and drop karo; ye XML khud likh deta hai.
- **Java/Kotlin code:** `new TextView(this)` se views banao aur layout me jodo. Dynamic UIs ke liye achha.
- **Jetpack Compose** (naya): XML ki jagah UI ko Kotlin functions ke roop me likho.

Common view attributes

Attribute	Matlab
android:id	Unique naam: `@+id/btnOk`. findViewById() ke saath use hota hai.
layout_width / layout_height	**match_parent** (parent bhar do), **wrap_content** (sirf zaroorat jitna), ya **dp** me size
android:text	Dikhne wala text (@string/name use karo)
textSize, textColor, textStyle	Size (**sp** me), colour, bold/italic
hint	Khali EditText me halka help text
padding / layout_margin	View ke andar ki jagah / view ke bahar ki jagah
background	View ke peeche ka colour ya drawable
gravity / layout_gravity	View ke andar content ki position / parent ke andar view ki position
visibility	visible, invisible (chhupa par jagah leta hai), gone (chhupa, jagah nahi)

Attribute	Matlab
onClick	Click hone par call hone wale method ka naam

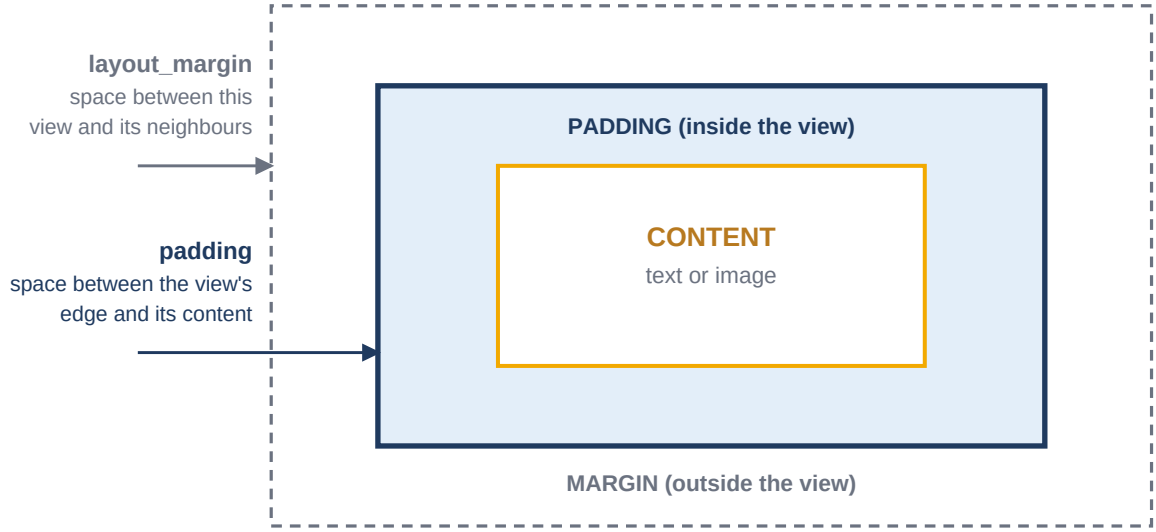


Figure 2: Margin, padding and content of a view

Unit	Full form aur use
dp (or dip)	Density-independent pixel. Sizes aur margins ke liye; sab screens par same size dikhta hai.
sp	Scale-independent pixel. **Text size** ke liye; user ki font-size setting follow karta hai.
px	Pixel. Avoid karo: screen density ke saath size badalta hai.

Events handle karna (view ko respond karwana)

- **findViewById(R.id.name)** XML ke view ko code ke variable se jodta hai.
- **setOnClickListener()** batata hai ki click par kya hoga. Anya listeners: **OnLongClickListener**, **OnCheckedChangeListener**, **OnItemSelectedListener**, **OnSeekBarChangeListener**.
- Purana shortcut XML me `android:onClick="submit"` aur activity me `public void submit(View v)` method hai.

Topic 2: User Interface Screen Elements (Widgets)

Android ready-made UI elements deta hai jinhe **widgets** kehte hain. Inhe unke kaam ke hisab se groups me baanta jaata hai.

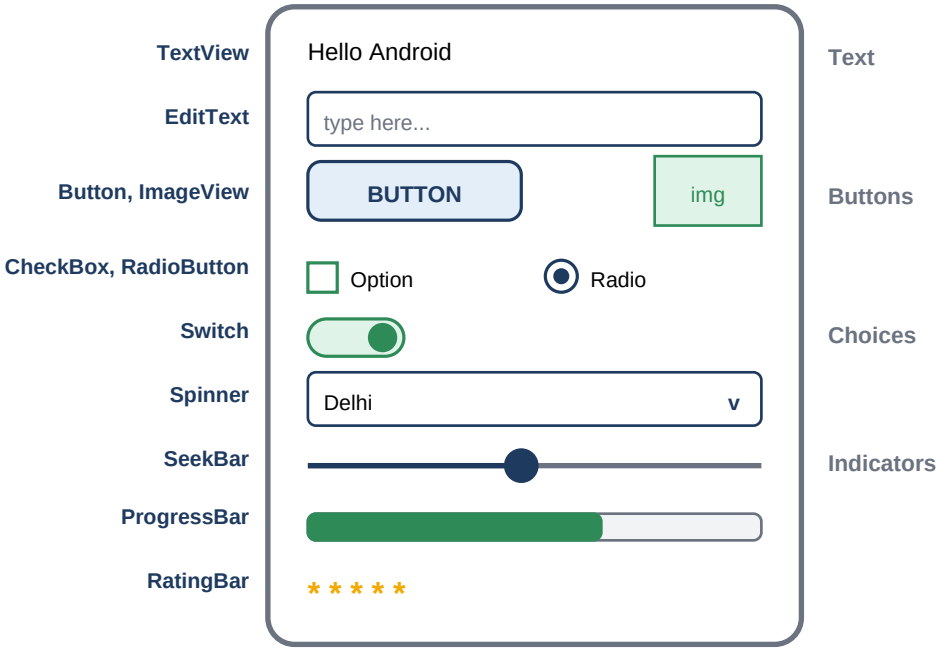


Figure 3: Common widgets on a screen

Group	Widgets
Text	TextView, EditText, AutoCompleteTextView
Buttons	Button, ImageButton, CheckBox, RadioButton / RadioGroup, ToggleButton, Switch
Indicators	ProgressBar, SeekBar, RatingBar
Pickers and clocks	DatePicker, TimePicker, TextClock, Chronometer
Adapter views	Spinner, ListView, GridView (RecyclerView is the modern list)
Other	ImageView, Toast, Snackbar, AlertDialog

2(a) Text widgets: TextView and EditText

- **TextView:** text dikhata hai; user ise edit nahi kar sakta. Attributes: text, textSize, textColor, textStyle, fontFamily, gravity, maxLines.
- **EditText:** text box jisme user type karta hai. Important attribute **inputType**: text, textPassword, number, phone, textEmailAddress, textMultiLine. Value `getText().toString()` se padho.
- **AutoCompleteTextView:** aisa EditText jo type karte samay suggestions dikhata hai (ArrayAdapter chahiye).

activity_main.xml: TextView, EditText and Button

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <TextView
        android:id="@+id/tvTitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Login"
        android:textSize="24sp"
        android:textStyle="bold" />

    <EditText
        android:id="@+id/etName"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter your name"
        android:inputType="textPersonName" />

    <Button
        android:id="@+id/btnSubmit"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Submit" />
</LinearLayout>
```

MainActivity.java (inside onCreate): handle the button click

```
EditText etName = findViewById(R.id.etName);
Button btnSubmit = findViewById(R.id.btnSubmit);

btnSubmit.setOnClickListener(v -> {
    String name = etName.getText().toString();
    Toast.makeText(this, "Hello " + name, Toast.LENGTH_SHORT).show();
});
```

2(b) Buttons: Button, CheckBox, RadioButton, ToggleButton, Switch

Widget	Use	Listener / method
Button	Tap karne par action karta hai	setOnClickListener()
ImageButton	Text ki jagah image wala button	setOnClickListener()
CheckBox	Do-state box; kai ek saath select ho sakte hain	setOnCheckedChangeListener(), isChecked()
RadioButton in a **RadioGroup**	Group me se sirf **ek** choice select hoti hai	RadioGroup.setOnCheckedChangeListener()
ToggleButton / **Switch**	ON / OFF control	setOnCheckedChangeListener(), isChecked()

XML: CheckBox and RadioGroup

```
<CheckBox android:id="@+id/cbAgree" android:layout_width="wrap_content"
    android:layout_height="wrap_content" android:text="I agree" />

<RadioGroup android:id="@+id/rgGender" android:layout_width="wrap_content"
    android:layout_height="wrap_content" android:orientation="horizontal">
    <RadioButton android:id="@+id/rbMale" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:text="Male" />
    <RadioButton android:id="@+id/rbFemale" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:text="Female" />
</RadioGroup>
```

Java: handle CheckBox and RadioGroup

```
CheckBox cb = findViewById(R.id.cbAgree);
cb.setOnCheckedChangeListener((buttonView, isChecked) ->
        Toast.makeText(this, isChecked ? "Agreed" : "Not agreed",
                       Toast.LENGTH_SHORT).show());

RadioGroup rg = findViewById(R.id.rgGender);
rg.setOnCheckedChangeListener((group, checkedId) -> {
    RadioButton rb = findViewById(checkedId);
    Toast.makeText(this, rb.getText(), Toast.LENGTH_SHORT).show();
});
```

2(c) Adapter views: Spinner and ListView

Real-life example

Adapter ek **waiter** jaisa hai: wo data (kitchen ki dishes ki list) leta hai aur har item ko screen par ready-made row me parosta hai.

- **AdapterView** data source se mile items ki list **Adapter** (ArrayAdapter, SimpleAdapter, CursorAdapter) ke zariye dikhata hai.
- **Spinner:** drop-down list; user ek item chunta hai. **ListView:** scroll hone wali vertical list. **GridView:** scroll hone wala grid.
- **RecyclerView** modern, tez list view hai aur naye apps ke liye prefer hota hai. ListView abhi bhi supported hai aur seekhna aasaan hai. **Gallery** API 16 me deprecated ho gaya.

Spinner data and XML

```
<!-- res/values/strings.xml -->
<string-array name="cities">
    <item>Delhi</item>
    <item>Mumbai</item>
    <item>Chennai</item>
    <item>Kolkata</item>
</string-array>

<!-- layout -->
<Spinner android:id="@+id/spCity" android:layout_width="match_parent"
    android:layout_height="wrap_content" />
```

Java: Spinner with ArrayAdapter

```
Spinner sp = findViewById(R.id.spCity);
ArrayAdapter<CharSequence> adapter = ArrayAdapter.createFromResource(
        this, R.array.cities, android.R.layout.simple_spinner_item);
adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
sp.setAdapter(adapter);

sp.setOnItemSelectedListener(new AdapterView.OnItemSelectedListener() {
    @Override
    public void onItemSelected(AdapterView<?> parent, View view, int pos, long id) {
        String city = parent.getItemAtPosition(pos).toString();
        Toast.makeText(MainActivity.this, city, Toast.LENGTH_SHORT).show();
    }
    @Override
    public void onNothingSelected(AdapterView<?> parent) { }
});
```

Java: ListView with ArrayAdapter

```
String[] items = {"Android", "Java", "Kotlin", "XML"};
ListView lv = findViewById(R.id.listView);          // <ListView android:id="@+id/listView" .../>
ArrayAdapter<String> ad = new ArrayAdapter<>(
        this, android.R.layout.simple_list_item_1, items);
lv.setAdapter(ad);
lv.setOnItemClickListener((parent, view, position, id) ->
        Toast.makeText(this, items[position], Toast.LENGTH_SHORT).show());
```

2(d) Indicators, pickers and other elements

Widget	Purpose	Key points
ProgressBar	Progress ya 'loading' dikhata hai	Circular (indeterminate) ya horizontal, max aur progress ke saath; setProgress()
SeekBar	Slider jise user drag karta hai	OnSeekBarChangeListener: onProgressChanged()
RatingBar	Star rating	numStars, stepSize, getRating()
DatePicker / TimePicker	Date / time chunna	DatePickerDialog aur TimePickerDialog inhe dialog me dikhate hain. **Month 0 se shuru** (January = 0)
TextClock / Chronometer	Current time / stopwatch dikhate hain	**AnalogClock** (API 23 me deprecated) aur **DigitalClock** (API 17 me deprecated) purane hain; TextClock use karo
ImageView	Image dikhata hai	src, scaleType (centerCrop, fitCenter), contentDescription
Toast	Kuch der ka chhota message	Toast.makeText(ctx, text, LENGTH_SHORT).show()
Snackbar	Neeche ka message, action ke saath ho sakta hai	Material library: Snackbar.make(view, text, LENGTH_LONG).show()
AlertDialog	Decision maangne wala pop-up	AlertDialog.Builder. **ProgressDialog** deprecated hai (API 26)

XML: horizontal ProgressBar

```
<ProgressBar android:id="@+id/progress" style="?android:attr/progressBarStyleHorizontal"
    android:layout_width="match_parent" android:layout_height="wrap_content"
    android:max="100" android:progress="40" />
```

Java: SeekBar, DatePickerDialog and AlertDialog

```
SeekBar seek = findViewById(R.id.seekBar);
seek.setOnSeekBarChangeListener(new SeekBar.OnSeekBarChangeListener() {
    @Override public void onProgressChanged(SeekBar bar, int progress, boolean fromUser) {
        textView.setText("Value: " + progress);
    }
    @Override public void onStartTrackingTouch(SeekBar bar) { }
    @Override public void onStopTrackingTouch(SeekBar bar) { }
});

// Date picker dialog (month 0 = January)
new DatePickerDialog(this, (view, year, month, day) ->
        textView.setText(day + "/" + (month + 1) + "/" + year), 2026, 0, 1).show();

// Alert dialog
new AlertDialog.Builder(this)
        .setTitle("Exit")
        .setMessage("Do you want to exit?")
        .setPositiveButton("Yes", (dialog, which) -> finish())
        .setNegativeButton("No", null)
        .show();
```

Topic 3: Designing User Interfaces with Layouts

Real-life example

Layout kisi **hall ka seating plan** hai. Ye decide karta hai ki har guest (view) kahan baithega: ek line me, kisi doosre guest ke relative, upar neeche stack, ya table me. Alag halls ko alag plans chahiye.

Definition

Layout ek **ViewGroup** hai jo screen par views ka structure aur arrangement define karta hai. Layouts **XML** (res/layout) me likhe jaate hain ya code me banaye jaate hain.

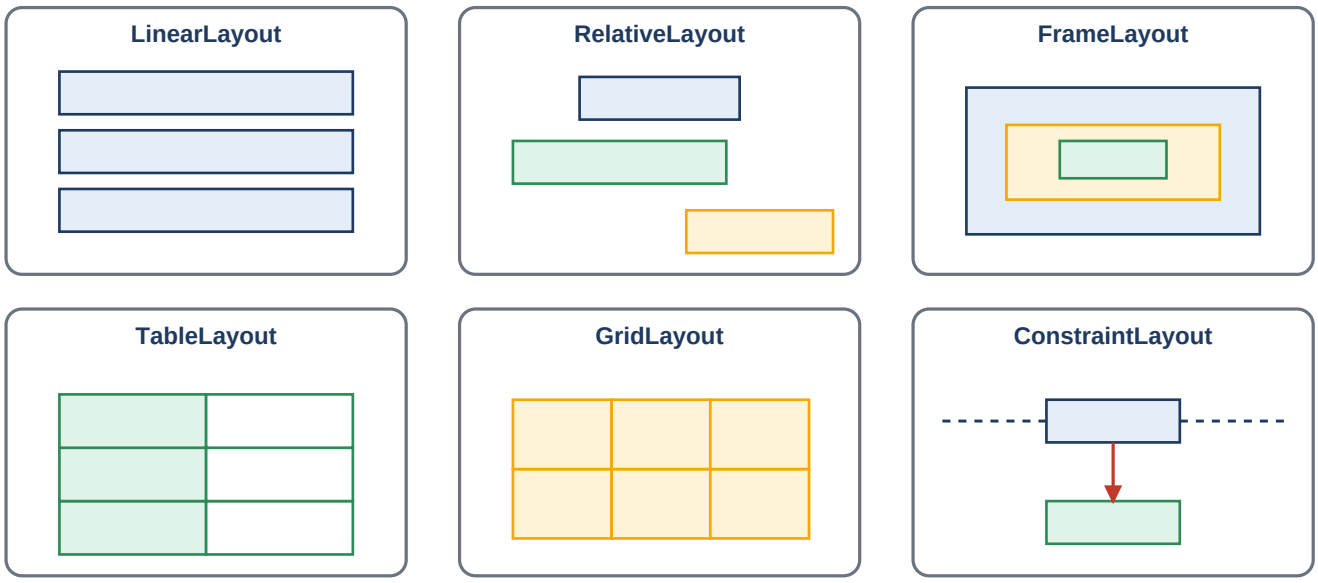


Figure 4: Main layout types

Layout	Children ko kaise arrange karta hai	Kis ke liye best
LinearLayout	Ek row ya column me (orientation)	Simple forms, row me buttons
RelativeLayout	Har view parent ya doosre views ke relative	Jahan positions ek doosre par depend hon
FrameLayout	Views ek ke upar ek stack	Ek view ya overlays (image par text)
TableLayout	Rows (TableRow) aur columns	Tabular data, label + field forms
GridLayout	Rows aur columns ka grid, spanning ke saath	Calculators, keypads
ConstraintLayout	Har view ke parent ya doosre views se constraints; flat hierarchy	Complex, responsive screens (Android Studio templates me default)

- **AbsoluteLayout** (fixed x, y positions) API level 3 se deprecated hai; use mat karo.
- **Layouts nest ho sakte hain**, par zyada nesting UI ko slow karti hai. Complex screens ke liye hierarchy flat rakhne ke liye ConstraintLayout prefer karo.
- Lambi screens ko **ScrollView** me wrap karo taaki user scroll kar sake (isme sirf ek direct child ho sakta hai).

3(a) LinearLayout

- Children ko ek direction me arrange karta hai: **android:orientation** = **vertical** (upar se neeche) ya **horizontal** (left se right). Default horizontal hai.
- **layout_weight** bachi hui jagah ko children me ratio me baantta hai. Weight use karte samay width (horizontal) ya height (vertical) **0dp** rakho.

- **gravity** view ke andar content ki position batati hai; **layout_gravity** view ki parent ke andar position batati hai.

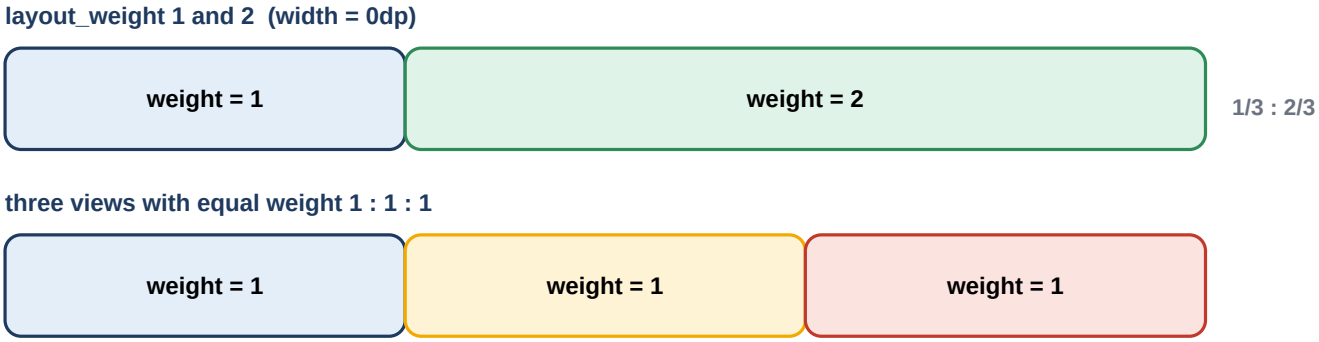


Figure 5: layout_weight in a horizontal LinearLayout

LinearLayout with weights 1 : 2 : 1

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal">

    <Button android:layout_width="0dp" android:layout_height="wrap_content"
        android:layout_weight="1" android:text="One" />
    <Button android:layout_width="0dp" android:layout_height="wrap_content"
        android:layout_weight="2" android:text="Two" />
    <Button android:layout_width="0dp" android:layout_height="wrap_content"
        android:layout_weight="1" android:text="Three" />
</LinearLayout>
```

3(b) RelativeLayout

- Har child **parent ke relative** (layout_alignParentTop, layout_alignParentEnd, layout_centerInParent) ya **kisi sibling view ke relative** uski id se (layout_below, layout_above, layout_toRightOf / layout_toEndOf, layout_alignLeft) position hota hai.
- Deep nesting ke bina flexible designs deta hai, par views sahi order me declare hone chahiye aur ids `@id/name` se refer hoti hain.

RelativeLayout: a label, a text box and a button

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="16dp">

    <TextView
        android:id="@+id/tvName"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_centerHorizontal="true"
        android:text="Name" />

    <EditText
        android:id="@+id/etName"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_below="@id/tvName" />

    <Button
        android:id="@+id/btnOk"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_below="@id/etName"
        android:layout_alignParentEnd="true"
        android:text="OK" />
</RelativeLayout>
```

3(c) FrameLayout, TableLayout, GridLayout

FrameLayout, TableLayout and GridLayout

```
<!-- FrameLayout: text placed over an image -->
<FrameLayout android:layout_width="match_parent" android:layout_height="200dp">
    <ImageView android:layout_width="match_parent" android:layout_height="match_parent"
        android:src="@drawable/photo" android:scaleType="centerCrop"
        android:contentDescription="Photo" />
    <TextView android:layout_width="wrap_content" android:layout_height="wrap_content"
        android:layout_gravity="bottom|center_horizontal"
        android:text="Caption" android:textColor="#FFFFFF" />
</FrameLayout>

<!-- TableLayout: label + field rows -->
<TableLayout android:layout_width="match_parent" android:layout_height="wrap_content"
    android:stretchColumns="1">
    <TableRow>
        <TextView android:text="Name:" />
        <EditText android:hint="Enter name" />
    </TableRow>
    <TableRow>
        <TextView android:text="Email:" />
        <EditText android:hint="Enter email" />
    </TableRow>
</TableLayout>

<!-- GridLayout: 2 columns -->
<GridLayout android:layout_width="match_parent" android:layout_height="wrap_content"
    android:columnCount="2">
    <Button android:text="1" /> <Button android:text="2" />
    <Button android:text="3" /> <Button android:text="4" />
</GridLayout>
```

- **FrameLayout:** aakhri child sabse upar draw hota hai. Har child ko place karne ke liye **layout_gravity** use karo.
- **TableLayout:** har **TableRow** ek row hai; columns ki sankhya sabse badi row jitni hoti hai. **stretchColumns** column ko bachi chaudai tak failata hai.
- **GridLayout: columnCount** aur **rowCount** grid define karte hain; children cells span kar sakte hain.

3(d) ConstraintLayout

- Har view ko **kam se kam ek horizontal aur ek vertical constraint** chahiye, jaise `layout_constraintTop_toTopOf="parent"`.
- Yahan **0dp** ka matlab 'match constraints' hai (constraints ke beech ki jagah bharo).
- **Flat hierarchy** deta hai, isliye complex screens tez chalti hain; Layout Editor drag karke constraints bana sakta hai.

ConstraintLayout

```
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent" android:layout_height="match_parent">

    <TextView android:id="@+id/tvTitle"
        android:layout_width="wrap_content" android:layout_height="wrap_content"
        android:text="Title"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

    <Button android:id="@+id/btnGo"
        android:layout_width="0dp" android:layout_height="wrap_content"
        android:text="Go"
        app:layout_constraintTop_toBottomOf="@id/tvTitle"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

UI created in Java code

```
// Creating a UI in code instead of XML
LinearLayout ll = new LinearLayout(this);
ll.setOrientation(LinearLayout.VERTICAL);

TextView tv = new TextView(this);
tv.setText("Hello from code");
Button btn = new Button(this);
btn.setText("Click");

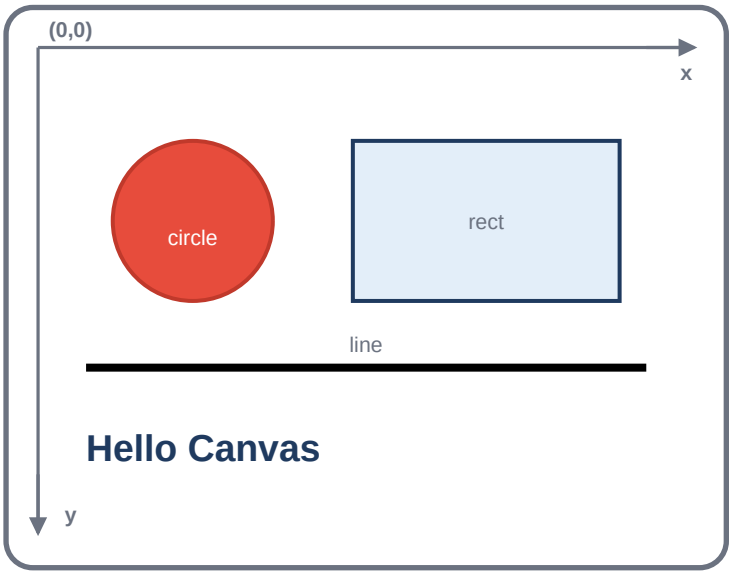
ll.addView(tv);
ll.addView(btn);
setContentView(ll);
```

View system (XML)	Jetpack Compose (Kotlin)
LinearLayout vertical / horizontal	Column / Row
FrameLayout	Box
ConstraintLayout	ConstraintLayout (Compose version)
TextView / EditText / Button / ImageView	Text / TextField / Button / Image
RecyclerView / ListView	LazyColumn / LazyRow
Design is in XML files; code uses findViewById	Design is written in Kotlin functions marked @Composable

Topic 4: Drawing: Text, Bitmaps and Shapes

Real-life example

Android me drawing **canvas par painting** jaisi hai. Tumhe chahiye ek **canvas** (surface), **brush aur colours** (Paint), aur kuch paint karne ko (shapes, text, pictures).



CANVAS draws, PAINT styles

- drawCircle(cx, cy, radius, paint)
- drawRect(left, top, right, bottom, paint)
- drawLine(x1, y1, x2, y2, paint)
- drawText(text, x, y, paint)
- drawBitmap(bitmap, x, y, paint)
- drawColor(color)

Figure 6: Canvas coordinates and drawing methods

Class	Role
Canvas	Drawing methods deta hai: drawCircle(), drawRect(), drawLine(), drawText(), drawBitmap(), drawColor()
Paint	Batata hai kaise draw karna hai: colour, style (FILL, STROKE), stroke width, text size, anti-aliasing
Bitmap	Pixel image jis par ya jise draw kiya jaata hai
Path	Lines aur curves se bani custom outline
Drawable	Jo kuch draw ho sake: bitmap, shape, vector, animation. src ya background ki tarah use hota hai

- Coordinate origin **(0, 0) top-left corner** hai; x right ki taraf aur y **neeche ki taraf** badhta hai. Units pixels hain.

4(a) Working with text

- **Layouts me:** TextView ko **textSize** (sp), **textColor**, **textStyle** (bold, italic) aur **fontFamily** (jaise sans-serif-medium ya res/font ka custom font) se style karo.
- **Code me:** `canvas.drawText("Hello", x, y, paint)` aur `paint.setTextSize()`. Custom font `Typeface.createFromAsset()` ya **res/font** se load hota hai.

4(b) Working with bitmaps

XML: show an image

```
<ImageView android:layout_width="200dp" android:layout_height="200dp"
    android:src="@drawable/photo" android:scaleType="centerCrop"
    android:contentDescription="My photo" />
```

Java: load, resize and draw a bitmap

```
Bitmap bmp = BitmapFactory.decodeResource(getResources(), R.drawable.photo);
imageView.setImageBitmap(bmp);                              // show it
Bitmap small = Bitmap.createScaledBitmap(bmp, 200, 200, true); // resize
canvas.drawBitmap(bmp, 50, 50, null);                       // draw on a canvas (in onDraw)
```

- Bade bitmaps bahut memory lete hain aur **OutOfMemoryError** la sakte hain. **inSampleSize** se chhota version load karo ya image library (**Glide, Coil, Picasso**) use karo.
- **scaleType** options: fitCenter, centerCrop, centerInside, fitXY.

4(c) Working with shapes

XML shape drawable (res/drawable): images ke bina rounded buttons, circles aur gradients banane ka aasaan tareeka. **Shape rectangle, oval, line ya ring** ho sakti hai.

A shape drawable and how to use it

```
<?xml version="1.0" encoding="utf-8"?>
<!-- res/drawable/rounded_box.xml -->
<shape xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="rectangle">
    <solid android:color="#1F3A5F" />
    <corners android:radius="12dp" />
    <stroke android:width="2dp" android:color="#F2A900" />
    <padding android:left="16dp" android:top="8dp" android:right="16dp" android:bottom="8dp" />
</shape>

<!-- gradient option, instead of <solid> -->
<!-- <gradient android:startColor="#1F3A5F" android:endColor="#0E7C86" android:angle="45" /> -->

<!-- use it -->
<TextView android:layout_width="wrap_content" android:layout_height="wrap_content"
    android:background="@drawable/rounded_box" android:text="Hello" android:textColor="#FFFFFF" />
```

- **Shape ke andar tags:** solid (fill colour), stroke (border), corners (rounded), gradient, padding, size.
- **Tip:** naye Material themes me simple Button `android:background` ko ignore kar sakta hai. `backgroundTint` use karo ya shape ko TextView / layout par lagao.
- **Code me:** `ShapeDrawable sd = new ShapeDrawable(new OvalShape()); sd.getPaint().setColor(Color.RED); view.setBackground(sd);`

Custom View (onDraw) se custom drawing

DrawView.java: drawing a circle, rectangle, line and text

```
public class DrawView extends View {
    private final Paint paint = new Paint(Paint.ANTI_ALIAS_FLAG);

    public DrawView(Context context) { super(context); }
    public DrawView(Context context, AttributeSet attrs) { super(context, attrs); }

    @Override
    protected void onDraw(Canvas canvas) {
        super.onDraw(canvas);
        canvas.drawColor(Color.WHITE);                  // background

        paint.setColor(Color.RED);
        paint.setStyle(Paint.Style.FILL);
        canvas.drawCircle(200, 200, 100, paint);        // cx, cy, radius

        paint.setColor(Color.BLUE);
        canvas.drawRect(100, 350, 400, 500, paint);     // left, top, right, bottom

        paint.setColor(Color.GREEN);
        paint.setStrokeWidth(8);
        canvas.drawLine(100, 600, 500, 650, paint);     // x1, y1, x2, y2

        paint.setColor(Color.BLACK);
        paint.setTextSize(48);
        canvas.drawText("Hello Canvas", 100, 800, paint);
    }
}

// In the activity:   setContentView(new DrawView(this));
// Or in XML:         <com.example.myapp.DrawView android:layout_width="match_parent"
//                        android:layout_height="match_parent" />
```

- **onDraw()** system tab call karta hai jab view ko draw karna ho; dobara draw karwane ke liye `invalidate()` call karo.
- **Paint** ko ek baar (field ki tarah) banao, onDraw() ke andar nahi, kyunki onDraw() bahut baar chalta hai.
- Do constructors isliye chahiye ki view code me bhi ban sake aur XML se inflate bhi ho sake.

Topic 5: Working with Animation

Real-life example

Animation **flip-book** jaisi hai: thodi thodi alag kai pictures jaldi jaldi dikhane se movement lagta hai. Android beech ki **in-between pictures** ka kaam bhi khud kar sakta hai.

Definition

Animation UI element me samay ke saath motion ya badlaav laane ki process hai, jaise view ko fade, move, rotate ya zoom karna.

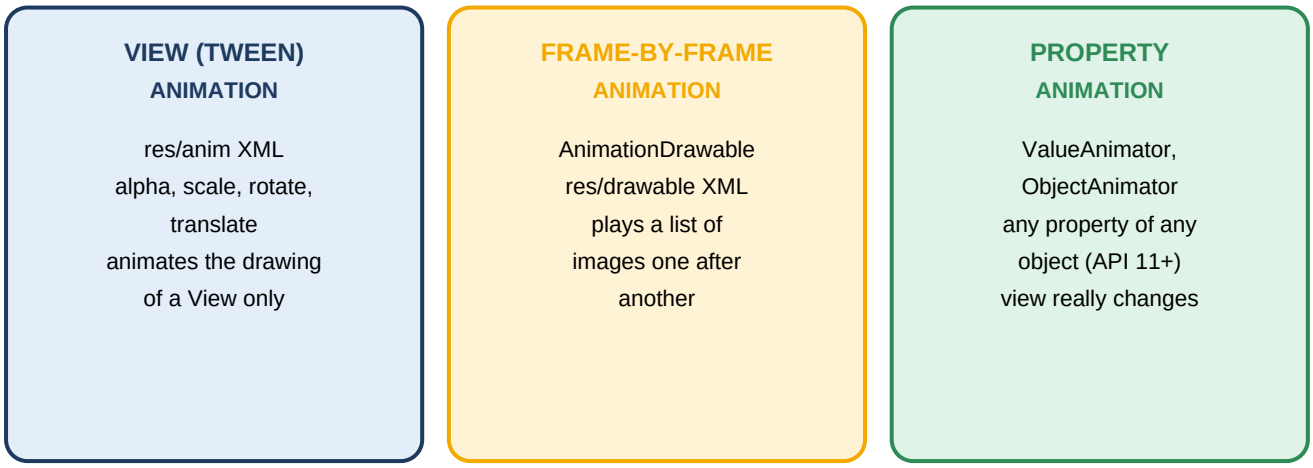


Figure 7: Types of animation in Android

Point	View (tween) animation	Frame-by-frame	Property animation
Idea	View kaise draw ho ye badalta hai	Images ki series dikhata hai	Object ki real values badalta hai
Classes	Animation, AnimationUtils	AnimationDrawable	ObjectAnimator, ValueAnimator, AnimatorSet
Kahan define	res/anim (XML) or code	res/drawable (animation-list)	Code (ya res/animator)
Since	API 1	API 1	API 11

5(a) View (tween) animation

Tween ka matlab 'beech ka'. Tum **start aur end values** dete ho aur system beech ke saare frames khud banata hai. Chaar basic transformations hain.

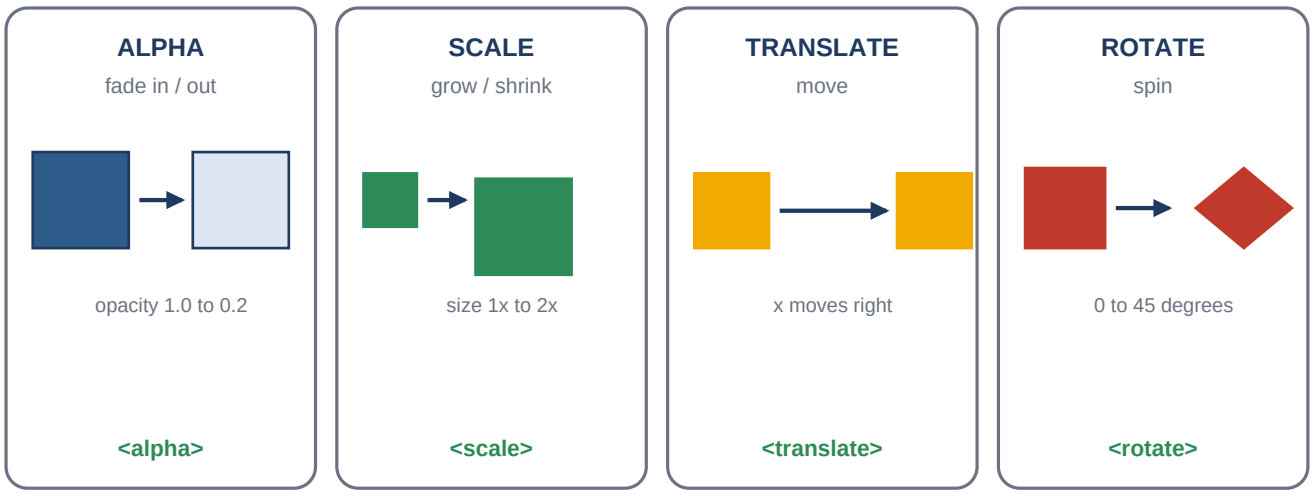


Figure 8: The four tween animations

XML tag	Effect	Main attributes
<alpha>	Fade in / fade out (transparency)	fromAlpha, toAlpha (0.0 to 1.0)
<scale>	Zoom in / zoom out	fromXScale, toXScale, fromYScale, toYScale, pivotX, pivotY
<translate>	View ko move karna	fromXDelta, toXDelta, fromYDelta, toYDelta
<rotate>	Pivot ke around ghumana	fromDegrees, toDegrees, pivotX, pivotY
<set>	Kai animations ko jodna	shareInterpolator, and children

- **Sab ke common attributes: duration** (ms), **startOffset** (delay), **repeatCount** (-1 ya 'infinite' = hamesha), **repeatMode** (restart ya reverse), **fillAfter** (end state rakho), **interpolator** (speed curve).

Tween animation XML files (res/anim)

```
<!-- res/anim/fade_in.xml -->
<alpha xmlns:android="http://schemas.android.com/apk/res/android"
    android:fromAlpha="0.0" android:toAlpha="1.0" android:duration="1000" />

<!-- res/anim/rotate.xml -->
<rotate xmlns:android="http://schemas.android.com/apk/res/android"
    android:fromDegrees="0" android:toDegrees="360"
    android:pivotX="50%" android:pivotY="50%" android:duration="1000" />

<!-- res/anim/zoom.xml -->
<scale xmlns:android="http://schemas.android.com/apk/res/android"
    android:fromXScale="1.0" android:toXScale="2.0"
    android:fromYScale="1.0" android:toYScale="2.0"
    android:pivotX="50%" android:pivotY="50%" android:duration="1000" />

<!-- res/anim/move.xml -->
<translate xmlns:android="http://schemas.android.com/apk/res/android"
    android:fromXDelta="0" android:toXDelta="300" android:duration="1000" />

<!-- res/anim/combo.xml: several together -->
<set xmlns:android="http://schemas.android.com/apk/res/android"
    android:interpolator="@android:anim/accelerate_decelerate_interpolator"
    android:fillAfter="true">
    <scale android:fromXScale="1.0" android:toXScale="1.5"
        android:fromYScale="1.0" android:toYScale="1.5"
        android:pivotX="50%" android:pivotY="50%" android:duration="800" />
    <alpha android:fromAlpha="1.0" android:toAlpha="0.4" android:duration="800" />
</set>
```

Java: load and start a tween animation

```
ImageView img = findViewById(R.id.imageView);
Button btn = findViewById(R.id.btnFade);

btn.setOnClickListener(v -> {
    Animation anim = AnimationUtils.loadAnimation(this, R.anim.fade_in);
    img.startAnimation(anim);
});

// Listen to the animation
anim.setAnimationListener(new Animation.AnimationListener() {
    @Override public void onAnimationStart(Animation a) { }
    @Override public void onAnimationEnd(Animation a) { /* done */ }
    @Override public void onAnimationRepeat(Animation a) { }
});

// Same thing in code, without XML
AlphaAnimation fade = new AlphaAnimation(0f, 1f);
fade.setDuration(1000);
img.startAnimation(fade);
```

Interpolator	Speed pattern
LinearInterpolator	Constant speed
AccelerateInterpolator	Dheere shuru, tez khatam
DecelerateInterpolator	Tez shuru, dheere khatam
AccelerateDecelerateInterpolator	Dheere, tez, phir dheere
BounceInterpolator	End par uchhalta hai
OvershootInterpolator	End se aage jaakar wapas aata hai

5(b) Frame-by-frame animation

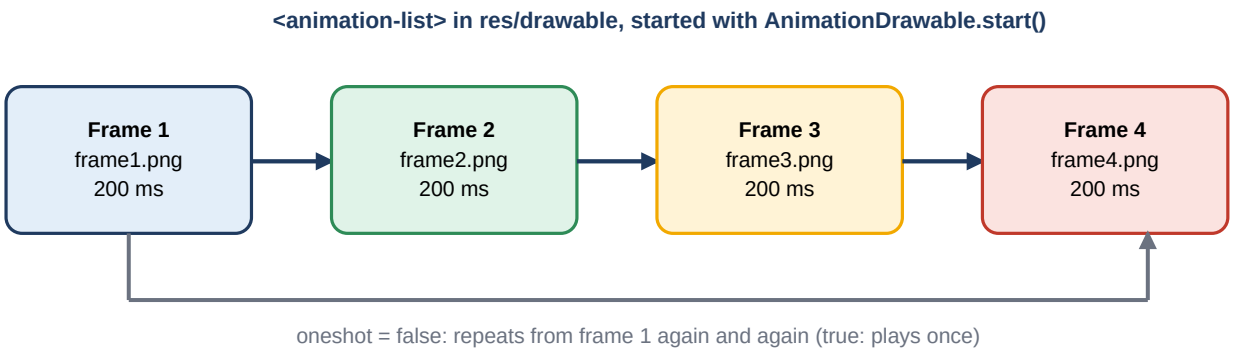


Figure 9: Frame-by-frame animation

- Kai images ek ke baad ek, har ek tay samay tak dikhti hain. **res/drawable** me **animation-list** XML file me define hoti hai aur **AnimationDrawable** se chalti hai.
- **android:oneshot**: true = ek baar, false = hamesha repeat. Har **item** me drawable aur ms me duration hota hai.

Frame animation XML

```
<!-- res/drawable/frame_anim.xml -->
<animation-list xmlns:android="http://schemas.android.com/apk/res/android"
    android:oneshot="false">
    <item android:drawable="@drawable/frame1" android:duration="200" />
    <item android:drawable="@drawable/frame2" android:duration="200" />
    <item android:drawable="@drawable/frame3" android:duration="200" />
</animation-list>
```

Java: play the frame animation

```
ImageView img = findViewById(R.id.imgAnim);
img.setBackgroundResource(R.drawable.frame_anim);
AnimationDrawable frameAnim = (AnimationDrawable) img.getBackground();

btnStart.setOnClickListener(v -> frameAnim.start());   // start the animation
btnStop.setOnClickListener(v -> frameAnim.stop());     // stop it
```

Common mistake

`frameAnim.start()` ko `onCreate()` ke andar call **mat** karo: drawable abhi window se attach nahi hota, isliye animation nahi chalti. Use button click se ya `onWindowFocusChanged(true)` se start karo.

5(c) Property animation

- **API 11** me aaya. Ye object ki **asli property** (translationX, rotation, alpha, scaleX) badalta hai, isliye view sach me move hota hai aur uska click area bhi saath move hota hai.
- **ObjectAnimator** naam wali property animate karta hai; **ValueAnimator** ek value animate karta hai jise tum khud use karte ho; **AnimatorSet** animators ko ek saath ya sequence me chalata hai.

Property animation examples

```
// Move the view 300 pixels to the right in 1 second
ObjectAnimator moveX = ObjectAnimator.ofFloat(view, "translationX", 0f, 300f);
moveX.setDuration(1000);
moveX.start();

// Rotate and fade together
ObjectAnimator rotate = ObjectAnimator.ofFloat(view, "rotation", 0f, 360f);
ObjectAnimator fade   = ObjectAnimator.ofFloat(view, "alpha", 1f, 0.2f);
AnimatorSet set = new AnimatorSet();
set.playTogether(rotate, fade);          // or set.playSequentially(rotate, fade);
set.setDuration(1500);
set.start();

// Short form with ViewPropertyAnimator
view.animate().translationX(300f).alpha(0.5f).setDuration(1000).start();

// ValueAnimator: count a progress bar from 0 to 100
ValueAnimator va = ValueAnimator.ofInt(0, 100);
va.setDuration(2000);
va.addUpdateListener(a -> progressBar.setProgress((int) a.getAnimatedValue()));
va.start();
```

Point	View (tween) animation	Property animation
Kya badalta hai	Sirf view kaise draw hota hai	Object ki asli property
Move ke baad click area	Original jagah par rehta hai	View ke saath move hota hai
Kis par chalta hai	Sirf views	Property wale kisi bhi object par
Since	API 1	API 11

- **Screen transitions:** purane code me startActivity() ke baad `overridePendingTransition(R.anim.fade_in, R.anim.fade_out)` dikhta hai; naye Android versions me isi kaam ke liye **overrideActivityTransition()** aata hai.
- **Tips:** animations chhoti rakho (200 se 500 ms), zyada use mat karo, aur un users ka dhyan rakho jinhone settings me animations off kiye hain.

PART B: Important Questions aur Model Answers (Hinglish)

Priority: **Very Likely** = BTEUP me sabse zyada aata hai. **Likely** = achha chance. **Possible** = aa sakta hai. Har answer exam format me hai: **Definition, Points, Diagram/Table/Code, Conclusion**.

Q1. View aur ViewGroup kya hain? UI hierarchy aur UI banane ke tareeke explain karo. — LIKELY

Definition / Parichay: View user interface ka basic building block hai; ye screen par kuch draw karta hai aur user events handle karta hai. ViewGroup ek invisible container hai jo doosre views ko rakhta aur arrange karta hai.

Points

- **Examples:** View: TextView, Button, ImageView. ViewGroup: LinearLayout, RelativeLayout, ConstraintLayout.
- **UI hierarchy:** screen ek tree hai. Root ek ViewGroup hota hai, jisme views aur doosre ViewGroups hote hain.
- **UI banane ke tareeke:** (1) XML layout files, (2) Layout Editor, (3) Java/Kotlin code, (4) Jetpack Compose.
- **Jodna:** setContentView(R.layout.activity_main) layout dikhata hai; findViewById(R.id.name) view deta hai; setOnClickListener() events handle karta hai.
- **Units:** sizes ke liye dp, text ke liye sp, px se bacho.

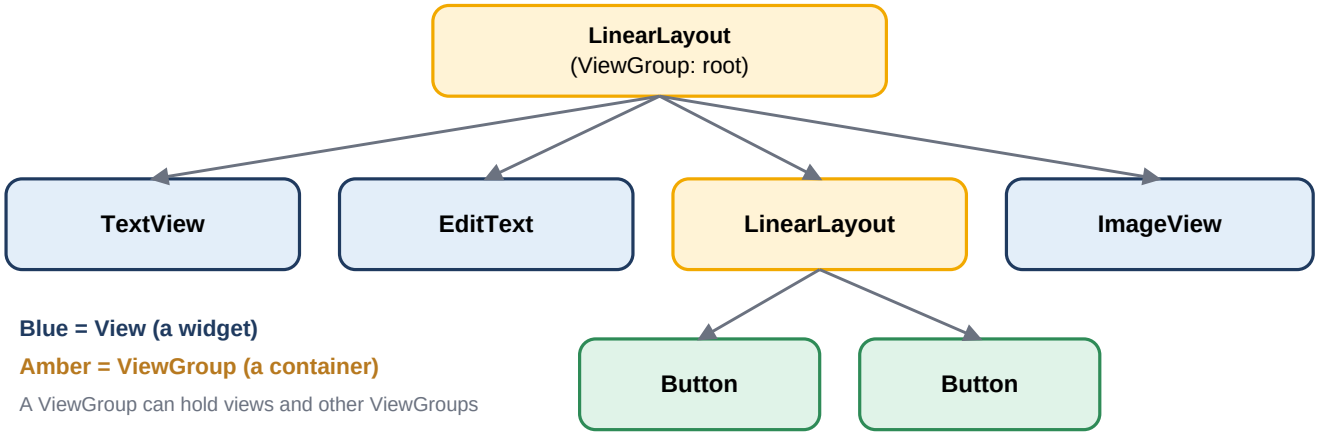


Figure: View hierarchy

Conclusion: View hierarchy samajhna har Android UI design ki buniyaad hai.

Q2. TextView aur EditText ko unke important attributes aur example ke saath explain karo.

VERY LIKELY

Definition / Parichay: TextView user ko text dikhata hai aur edit karne nahi deta. EditText, TextView ki subclass hai jo user ko text type aur edit karne deta hai.

Points

- **TextView attributes:** text, textSize (sp), textColor, textStyle (bold, italic), fontFamily, gravity, maxLines.
- **EditText attributes:** hint (help text), inputType (text, textPassword, number, phone, textEmailAddress), maxLength, lines.
- User ka input `getText().toString()` se padha jaata hai.
- **AutoCompleteTextView** aisa EditText hai jo type karte samay suggestions dikhata hai.

TextView and EditText

```
<TextView android:id="@+id/tvTitle" android:layout_width="wrap_content"
    android:layout_height="wrap_content" android:text="Login"
    android:textSize="24sp" android:textStyle="bold" />

<EditText android:id="@+id/etName" android:layout_width="match_parent"
    android:layout_height="wrap_content" android:hint="Enter your name"
    android:inputType="textPersonName" />

// Java
EditText etName = findViewById(R.id.etName);
String name = etName.getText().toString();
Toast.makeText(this, "Hello " + name, Toast.LENGTH_SHORT).show();
```

Conclusion: TextView aur EditText text dikhane aur lene ke sabse zyada use hone wale widgets hain.

Q3. Button, CheckBox, RadioButton aur Switch ko example program ke saath explain karo.

VERY LIKELY

Definition / Parichay: Ye wo widgets hain jinse users actions karte hain aur choices chunte hain.

Points

- **Button:** tap karne par action karta hai; `setOnClickListener()` se handle hota hai. **ImageButton** image dikhata hai.
- **CheckBox:** do-state box; kai select ho sakte hain. `isChecked()` ya `setOnCheckedChangeListener()` use karo.
- **RadioButton: RadioGroup** ke andar rakha jaata hai; group me se sirf ek choice select hoti hai.
- **ToggleButton / Switch:** ON/OFF control, `setOnCheckedChangeListener()` ke saath.

Handling Button, CheckBox and RadioGroup

```
CheckBox cb = findViewById(R.id.cbAgree);
cb.setOnCheckedChangeListener((buttonView, isChecked) ->
        Toast.makeText(this, isChecked ? "Agreed" : "Not agreed",
                       Toast.LENGTH_SHORT).show());

RadioGroup rg = findViewById(R.id.rgGender);
rg.setOnCheckedChangeListener((group, checkedId) -> {
    RadioButton rb = findViewById(checkedId);
    Toast.makeText(this, rb.getText(), Toast.LENGTH_SHORT).show();
});

Button btn = findViewById(R.id.btnSubmit);
btn.setOnClickListener(v -> Toast.makeText(this, "Submitted",
                                           Toast.LENGTH_SHORT).show());
```

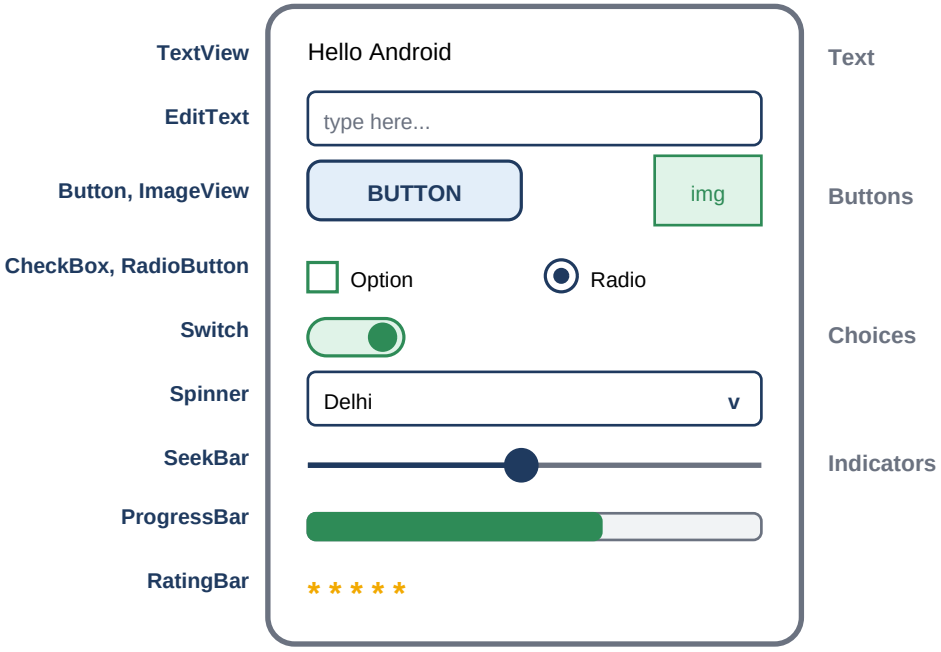


Figure: Common widgets

Conclusion: Har widget ka ek listener hota hai jo user ke action ki khabar program ko deta hai.

Q4. Spinner aur ListView ko Adapters ki madad se explain karo. Program likho.

VERY LIKELY

Definition / Parichay: Spinner aur ListView adapter views hain: ye data source se mile items ki list Adapter ke zariye dikhate hain. Spinner drop-down list hai aur ListView scroll hone wali list hai.

Points

- **Adapter:** data (array, list, database) ko view se jodta hai aur har item ke liye row banata hai. Types: ArrayAdapter, SimpleAdapter, CursorAdapter.
- **Spinner:** user ek item chunta hai; `setOnItemSelectedListener()` use karo.
- **ListView:** user item par tap karta hai; `setOnItemClickListener()` use karo.
- **RecyclerView** bade ya complex lists ka modern replacement hai.

ListView and Spinner with ArrayAdapter

```
String[] items = {"Android", "Java", "Kotlin", "XML"};
ListView lv = findViewById(R.id.listView);
ArrayAdapter<String> ad = new ArrayAdapter<>(
        this, android.R.layout.simple_list_item_1, items);
lv.setAdapter(ad);
lv.setOnItemClickListener((parent, view, position, id) ->
        Toast.makeText(this, items[position], Toast.LENGTH_SHORT).show());

Spinner sp = findViewById(R.id.spCity);
ArrayAdapter<CharSequence> adapter = ArrayAdapter.createFromResource(
        this, R.array.cities, android.R.layout.simple_spinner_item);
adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
sp.setAdapter(adapter);
```

Conclusion: Adapters data ko uske display se alag rakhte hain, isliye ek hi view kisi bhi tarah ka data dikha sakta hai.

Q5. ProgressBar, SeekBar, RatingBar, DatePicker aur TimePicker par short notes likho.

LIKELY

Definition / Parichay: Ye indicator aur picker widgets hain jo progress dikhane aur ratings, dates, times jaisi values lene ke kaam aate hain.

Points

- **ProgressBar:** progress ya loading dikhata hai; circular (indeterminate) ya horizontal, `max` aur `setProgress()` ke saath.
- **SeekBar:** slider jise user drag karta hai; `OnSeekBarChangeListener` onProgressChanged() me value deta hai.
- **RatingBar:** star rating; attributes numStars aur stepSize; `getRating()` rating deta hai.
- **DatePicker / TimePicker:** date ya time chunna; **DatePickerDialog** aur **TimePickerDialog** inhe dialog me dikhate hain. Date callback me **month 0 se shuru** hota hai.
- **TextClock** time dikhata hai; AnalogClock aur DigitalClock deprecated hain.

SeekBar and RatingBar

```
SeekBar seek = findViewById(R.id.seekBar);
seek.setOnSeekBarChangeListener(new SeekBar.OnSeekBarChangeListener() {
    @Override public void onProgressChanged(SeekBar bar, int progress, boolean fromUser) {
        textView.setText("Value: " + progress);
    }
    @Override public void onStartTrackingTouch(SeekBar bar) { }
    @Override public void onStopTrackingTouch(SeekBar bar) { }
});
RatingBar rb = findViewById(R.id.ratingBar);
float stars = rb.getRating();
```

Conclusion: Ye widgets data entry aur feedback ko aasaan aur visual banate hain.

Q6. Layout kya hai? Android me layouts ke types explain karo. VERY LIKELY

Definition / Parichay: Layout ek ViewGroup hai jo screen par views ka structure aur arrangement define karta hai. Layouts res/layout ki XML files me define hote hain ya code me banaye jaate hain.

Points

- **LinearLayout:** children ek row ya column me (orientation); weights support karta hai.
- **RelativeLayout:** har child parent ya sibling ke relative position hota hai.
- **FrameLayout:** children ek ke upar ek stack.
- **TableLayout:** TableRow se rows aur columns me children. **GridLayout:** rows aur columns ka grid.
- **ConstraintLayout:** constraints se positioned views; flat hierarchy; templates me default.
- **Common attributes:** layout_width aur layout_height (match_parent, wrap_content), padding, layout_margin, gravity. AbsoluteLayout deprecated hai.

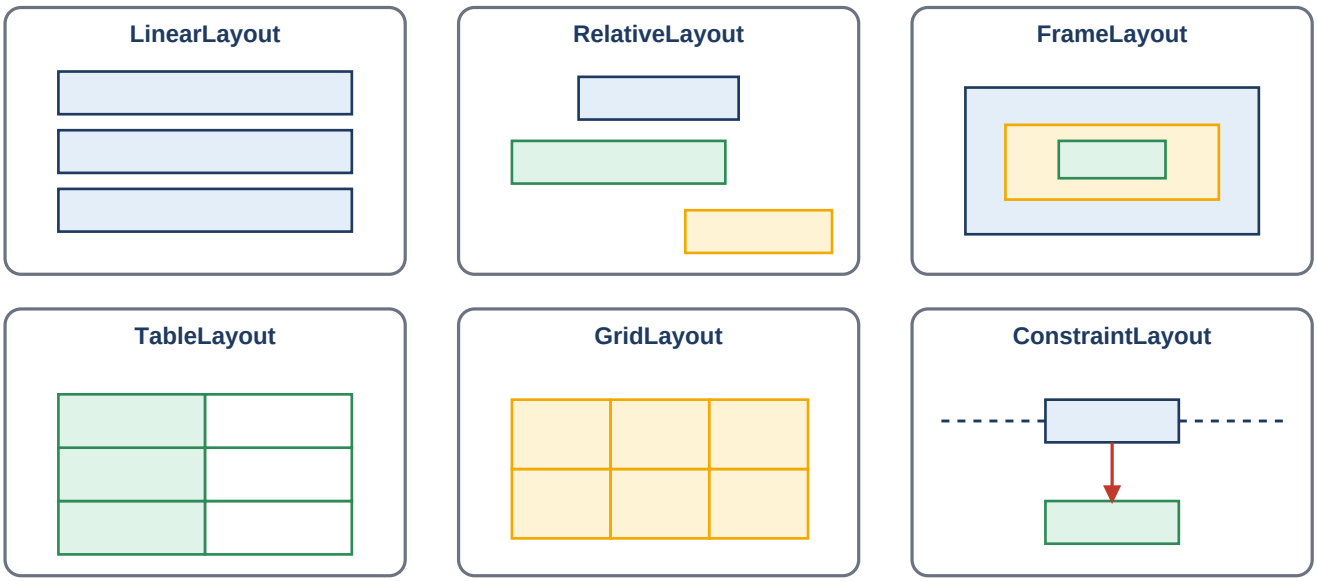


Figure: Types of layouts

Conclusion: Sahi layout chunne se clean, flexible aur tez user interface milta hai.

Q7. LinearLayout ko uske attributes ke saath explain karo aur XML example likho.

VERY LIKELY

Definition / Parichay: LinearLayout apne saare children ko ek hi direction me arrange karta hai, ya to vertically column me ya horizontally row me.

Points

- **android:orientation:** vertical (upar se neeche) ya horizontal (left se right, default).
- **android:layout_weight:** bachi hui jagah ko children me ratio me baantta hai; use karte samay size 0dp rakho.
- **android:gravity:** view ke andar content ki position. **android:layout_gravity:** view ki parent ke andar position.
- **padding / layout_margin:** view ke andar / bahar ki jagah.
- Layouts nest ho sakte hain, jaise vertical LinearLayout ke andar horizontal LinearLayouts.

LinearLayout: nested vertical and horizontal

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="match_parent"
    android:orientation="vertical" android:padding="16dp">

    <TextView android:layout_width="wrap_content" android:layout_height="wrap_content"
        android:text="Register" android:textSize="22sp" />

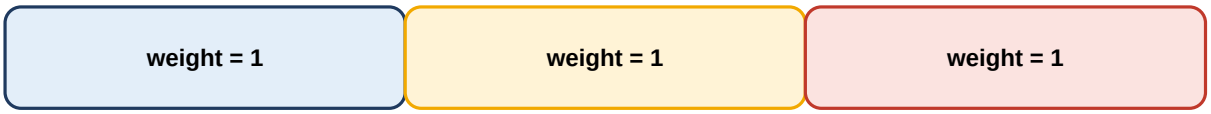
    <EditText android:layout_width="match_parent" android:layout_height="wrap_content"
        android:hint="Name" />

    <LinearLayout android:layout_width="match_parent" android:layout_height="wrap_content"
        android:orientation="horizontal">
        <Button android:layout_width="0dp" android:layout_height="wrap_content"
            android:layout_weight="1" android:text="Save" />
        <Button android:layout_width="0dp" android:layout_height="wrap_content"
            android:layout_weight="1" android:text="Cancel" />
    </LinearLayout>
</LinearLayout>
```

layout_weight 1 and 2 (width = 0dp)



three views with equal weight 1 : 1 : 1



Free space is shared in the ratio of the weights.

Figure: layout_weight

Conclusion: LinearLayout forms aur buttons ki rows ke liye sabse simple layout hai.

Q8. RelativeLayout ko uske important attributes aur example ke saath explain karo.

VERY LIKELY

Definition / Parichay: RelativeLayout har child ko parent layout ya doosre sibling views ke relative position karta hai, jisse deep nesting ke bina flexible designs milte hain.

Points

- **Parent ke relative:** layout_alignParentTop / Bottom / Start / End, layout_centerInParent, layout_centerHorizontal, layout_centerVertical.
- **Sibling ke relative (id se):** layout_above, layout_below, layout_toLeftOf, layout_toRightOf (ya Start/End), layout_alignLeft, layout_alignTop, layout_alignBaseline.
- Ids `@id/name` se refer hoti hain; jis view ko refer kiya ho wo pehle declare hona chahiye.

RelativeLayout example

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="match_parent"
    android:padding="16dp">

    <TextView android:id="@+id/tvName" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:layout_centerHorizontal="true"
        android:text="Name" />

    <EditText android:id="@+id/etName" android:layout_width="match_parent"
        android:layout_height="wrap_content" android:layout_below="@id/tvName" />

    <Button android:id="@+id/btnOk" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:layout_below="@id/etName"
        android:layout_alignParentEnd="true" android:text="OK" />
</RelativeLayout>
```

Conclusion: RelativeLayout tab kaam aata hai jab ek view ki position doosre par depend kare.

Q9. FrameLayout, TableLayout, GridLayout aur ConstraintLayout par short notes likho.

LIKELY

Definition / Parichay: LinearLayout aur RelativeLayout ke alawa Android special zarooraton ke liye ye layouts deta hai.

Points

- **FrameLayout:** children ko ek ke upar ek stack karta hai; aakhri child upar; layout_gravity use hoti hai. Overlays aur fragments ke liye.
- **TableLayout:** children ko rows (TableRow) aur columns me arrange karta hai; **stretchColumns** aur **shrinkColumns** width control karte hain.
- **GridLayout:** columnCount aur rowCount se bana grid; children cells span kar sakte hain.
- **ConstraintLayout:** har view ke parent ya siblings se horizontal aur vertical constraints; flat hierarchy; complex, responsive screens ke liye achha.

TableLayout example

```
<TableLayout android:layout_width="match_parent" android:layout_height="wrap_content"
    android:stretchColumns="1">
    <TableRow>
        <TextView android:text="Name:" />
        <EditText android:hint="Enter name" />
    </TableRow>
    <TableRow>
        <TextView android:text="Email:" />
        <EditText android:hint="Enter email" />
    </TableRow>
</TableLayout>
```

Layout	Arrangement	Best for
FrameLayout	Stack	Overlays
TableLayout	Rows, columns	Forms
GridLayout	Grid	Keypads
ConstraintLayout	Constraints	Complex screens

Conclusion: Har screen ke liye sahi layout use karne se code simple aur UI efficient rehta hai.

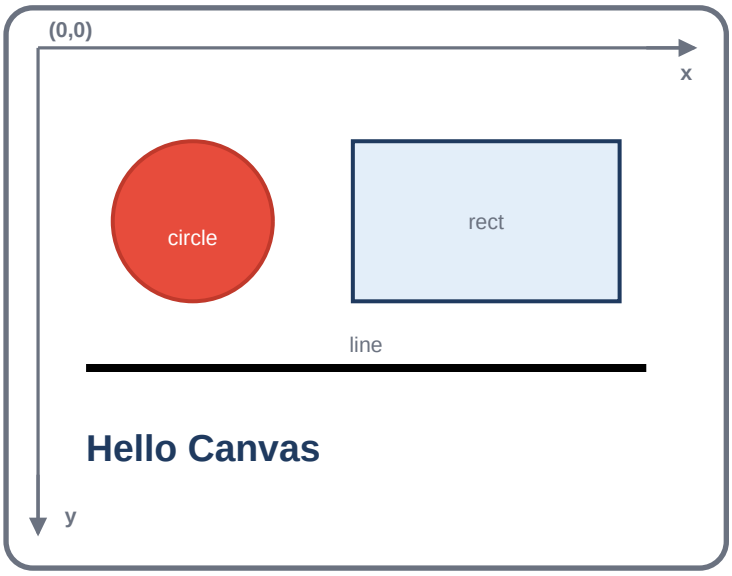
Q10. Android me drawing explain karo: Canvas, Paint, shapes, text aur bitmaps. Custom View ka program likho.

VERY LIKELY

Definition / Parichay: Android me drawing ka matlab screen par graphics banana hai. Iske liye Canvas (drawing surface), Paint (kaise draw karna hai) aur draw hone wali cheez (shape, text ya bitmap) chahiye.

Points

- **Canvas:** drawCircle(), drawRect(), drawLine(), drawText(), drawBitmap() methods deta hai. Origin (0, 0) top-left corner hai.
- **Paint:** colour, style (FILL ya STROKE), stroke width, text size, anti-aliasing.
- **Shapes:** code me draw hote hain, ya XML shape drawable (rectangle, oval, line, ring) me solid, stroke, corners aur gradient ke saath define hote hain.
- **Text:** Paint.setTextSize() ke saath canvas.drawText(); normal text ke liye TextView.
- **Bitmaps:** BitmapFactory.decodeResource() image load karta hai; ImageView ya canvas.drawBitmap() use dikhata hai; OutOfMemoryError se bachne ke liye bade images scale karo.



CANVAS draws, PAINT styles

- drawCircle(cx, cy, radius, paint)
- drawRect(left, top, right, bottom, paint)
- drawLine(x1, y1, x2, y2, paint)
- drawText(text, x, y, paint)
- drawBitmap(bitmap, x, y, paint)
- drawColor(color)

Figure: Canvas and drawing methods

Custom View with onDraw()

```
public class DrawView extends View {
    private final Paint paint = new Paint(Paint.ANTI_ALIAS_FLAG);

    public DrawView(Context context) { super(context); }

    @Override
    protected void onDraw(Canvas canvas) {
        super.onDraw(canvas);
        paint.setColor(Color.RED);
        canvas.drawCircle(200, 200, 100, paint);
        paint.setColor(Color.BLUE);
        canvas.drawRect(100, 350, 400, 500, paint);
        paint.setColor(Color.BLACK);
        paint.setTextSize(48);
        canvas.drawText("Hello Canvas", 100, 800, paint);
    }
}
// Activity: setContentView(new DrawView(this));
```

Conclusion: Canvas aur Paint screen par kya dikhe, is par poora control dete hain.

Q11. Animation kya hai? Android me animation ke types explain karo. VERY LIKELY

Definition / Parichay: Animation UI element me samay ke saath motion ya badlaav laane ki process hai, jaise view ko fade, move, rotate ya zoom karna. Isse app attractive aur samajhne me aasaan banta hai.

Points

- **View (tween) animation:** alpha, scale, translate aur rotate, XML (res/anim) ya code me define; ye sirf view kaise draw hota hai ye badalta hai.
- **Frame-by-frame animation:** animation-list aur AnimationDrawable se ek ke baad ek chalne wali images ki series.
- **Property animation (API 11):** ObjectAnimator, ValueAnimator aur AnimatorSet se asli property values badalta hai; kisi bhi object par chalta hai.
- **Interpolators** speed control karte hain: linear, accelerate, decelerate, bounce, overshoot.

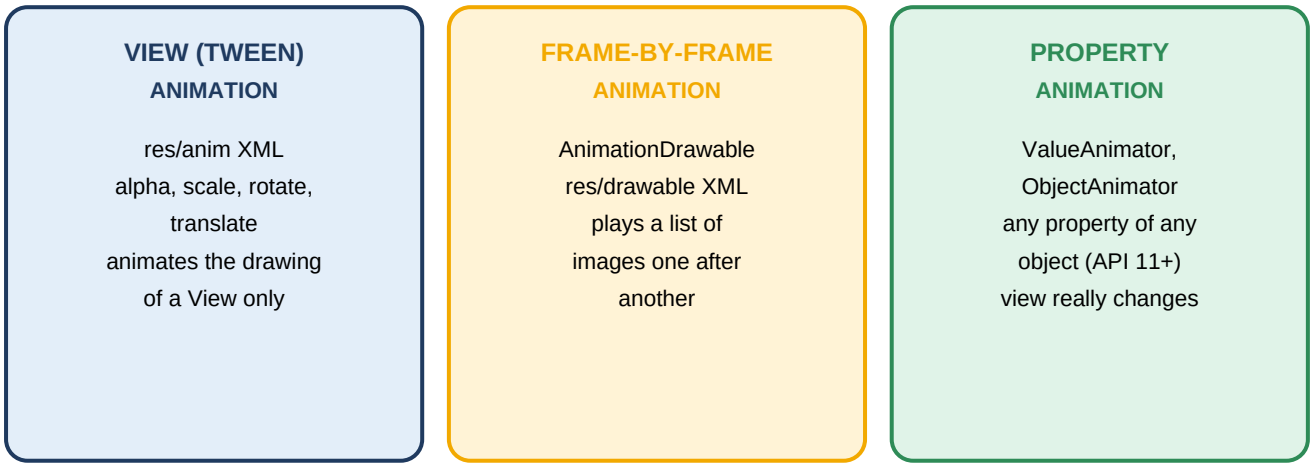


Figure: Types of animation

Conclusion: Android har tarah ki animation ke liye ek simple tool deta hai, chhote fade se lekar complex object movement tak.

Q12. Tween animation explain karo. Alpha, scale, translate aur rotate animations ka XML aur Java code likho.

VERY LIKELY

Definition / Parichay: Tween animation view par simple transformations lagata hai. Developer start aur end values deta hai aur system beech ke frames khud calculate karta hai.

Points

- **Alpha:** transparency badalta hai (fade in/out); fromAlpha aur toAlpha.
- **Scale:** zoom in/out; fromXScale, toXScale, fromYScale, toYScale, pivotX, pivotY.
- **Translate:** view ko move karta hai; fromXDelta, toXDelta, fromYDelta, toYDelta.
- **Rotate:** view ko ghumata hai; fromDegrees, toDegrees, pivotX, pivotY.
- **Common:** duration, startOffset, repeatCount, repeatMode, fillAfter, interpolator. Animations jodne ke liye **<set>** use karo.
- **Steps:** res/anim me XML banao, `AnimationUtils.loadAnimation()` se load karo, `view.startAnimation()` se start karo.

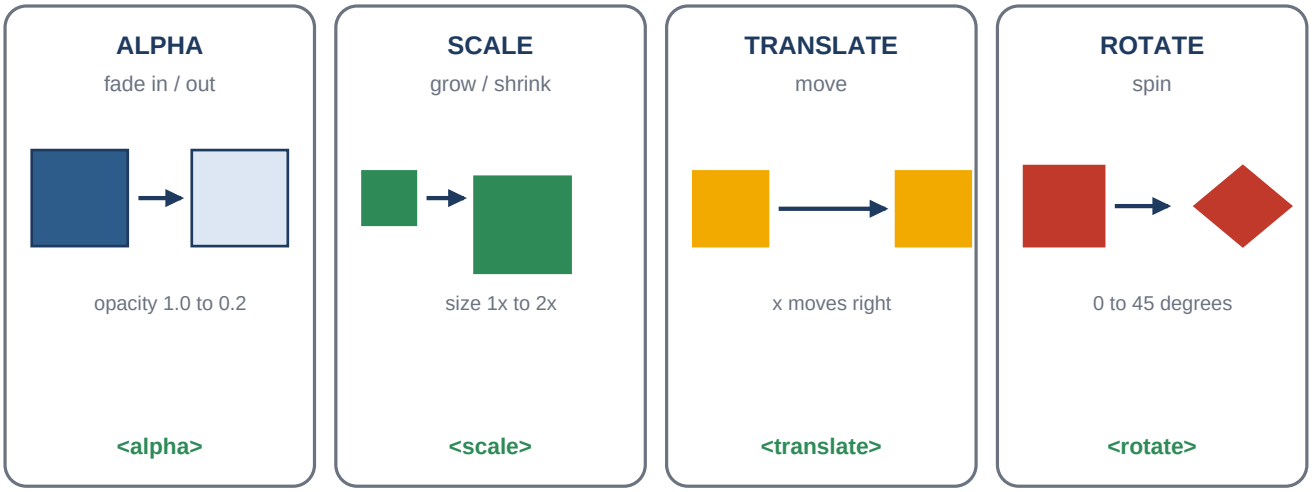


Figure: The four tween animations

Tween animations in XML

```
<!-- res/anim/fade_in.xml -->
<alpha xmlns:android="http://schemas.android.com/apk/res/android"
    android:fromAlpha="0.0" android:toAlpha="1.0" android:duration="1000" />

<!-- res/anim/zoom.xml -->
<scale xmlns:android="http://schemas.android.com/apk/res/android"
    android:fromXScale="1.0" android:toXScale="2.0"
    android:fromYScale="1.0" android:toYScale="2.0"
    android:pivotX="50%" android:pivotY="50%" android:duration="1000" />

<!-- res/anim/move.xml -->
<translate xmlns:android="http://schemas.android.com/apk/res/android"
    android:fromXDelta="0" android:toXDelta="300" android:duration="1000" />

<!-- res/anim/rotate.xml -->
<rotate xmlns:android="http://schemas.android.com/apk/res/android"
    android:fromDegrees="0" android:toDegrees="360"
    android:pivotX="50%" android:pivotY="50%" android:duration="1000" />
```

Java: start the animation

```
ImageView img = findViewById(R.id.imageView);
Animation anim = AnimationUtils.loadAnimation(this, R.anim.fade_in);
img.startAnimation(anim);
```

Conclusion: Tween animations fade, zoom, move aur rotate effects jodne ka sabse tez tareeka hain.

Q13. Frame-by-frame animation example ke saath explain karo. LIKELY

Definition / Parichay: Frame-by-frame animation images ka ek sequence ek ke baad ek, har ek tay samay tak dikhata hai, flip-book ki tarah. Ye animation-list se define hota hai aur AnimationDrawable se chalta hai.

Points

- Frame images (frame1.png, frame2.png, ...) **res/drawable** me rakho.
- Har frame ke liye **item** ke saath **animation-list** XML banao, jisme drawable aur duration (ms) ho. **oneshot** = false animation ko repeat karta hai.
- Use ImageView ka background banao, **AnimationDrawable** lo aur **start()** ya **stop()** call karo.
- Ise onCreate() me start mat karo; button click ya onWindowFocusChanged() se start karo.

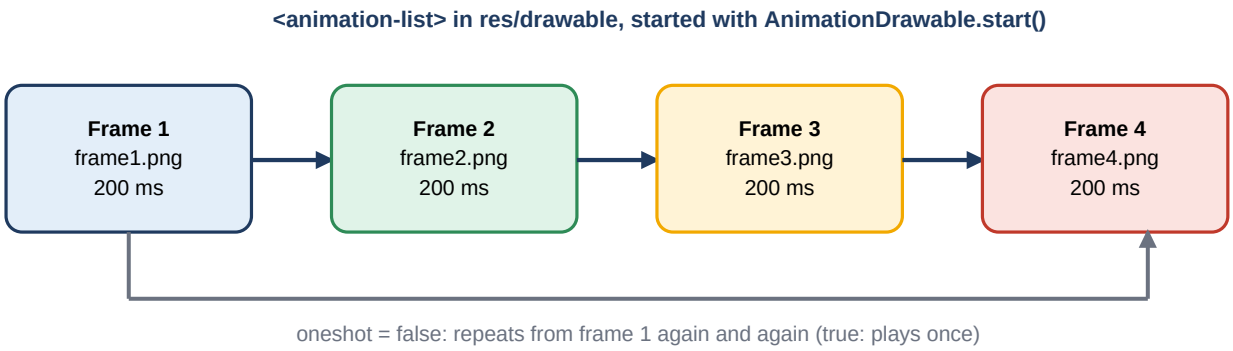


Figure: Frame-by-frame animation

Frame animation

```
<!-- res/drawable/frame_anim.xml -->
<animation-list xmlns:android="http://schemas.android.com/apk/res/android"
    android:oneshot="false">
    <item android:drawable="@drawable/frame1" android:duration="200" />
    <item android:drawable="@drawable/frame2" android:duration="200" />
    <item android:drawable="@drawable/frame3" android:duration="200" />
</animation-list>

// Java
ImageView img = findViewById(R.id.imgAnim);
img.setBackgroundResource(R.drawable.frame_anim);
AnimationDrawable frameAnim = (AnimationDrawable) img.getBackground();
btnStart.setOnClickListener(v -> frameAnim.start());
```

Conclusion: Frame animation chhoti, hand-drawn style animations, jaise loading icon, ke liye best hai.

Q14. Difference likho: (a) View animation aur Property animation, (b) dp aur sp, (c) match_parent aur wrap_content, (d) LinearLayout aur RelativeLayout.

POSSIBLE

Definition / Parichay: Ye jodiyan aksar confuse hoti hain. Tables inhe compare karti hain.

Points

- Har table dono items ko point by point compare karti hai.

(a) Point	View animation	Property animation
Badalta hai	Sirf view kaise draw hota hai	Object ki asli property
Click area	Purani jagah par	View ke saath move
Kis par	Sirf views	Koi bhi object
Since	API 1	API 11

(b) Point	dp	sp
Full form	Density-independent pixel	Scale-independent pixel
Use	Sizes, margins, padding	Text size
Font setting follow	Nahi	Haan

(c) Point	match_parent	wrap_content
Size	Parent jitna bada	Content jitna bada

(d) Point	LinearLayout	RelativeLayout
Arrangement	Ek row ya column	Parent ya siblings ke relative
Key attribute	orientation, layout_weight	layout_below, layout_toRightOf, alignParent...
Best for	Simple forms	Position doosre views par depend

Conclusion: In differences ko jaanne se har UI kaam ke liye sahi tool chunna aasaan hota hai.

Quick Revision Sheet (exam se pehle raat ko padho)

Term	Ye yaad rakho
View / ViewGroup	Widget / views ka container (layout)
Units	Size ke liye dp, text ke liye sp, px se bacho
layout_width values	match_parent, wrap_content, ya dp
visibility	visible, invisible (jagah rakhta hai), gone (jagah nahi)
Event handling	findViewById + setOnClickListener
EditText	inputType, hint, getText().toString()
RadioButton	RadioGroup ke andar: sirf ek select
Spinner / ListView	Adapter views; ArrayAdapter; OnItemSelected / OnItemClick
Date picker month	0 se shuru (January = 0)
Deprecated	AnalogClock, DigitalClock, Gallery, ProgressDialog, AbsoluteLayout
LinearLayout	orientation, layout_weight (0dp), gravity vs layout_gravity
RelativeLayout	layout_below, layout_toRightOf, layout_alignParent..., layout_centerInParent
Frame / Table / Grid	Stack / rows (TableRow) / columnCount
ConstraintLayout	Parent ya siblings se constraints; flat; 0dp = match constraints
Canvas / Paint	Surface + methods / colour, style, size. Origin top-left
Shape drawable	shape = rectangle, oval, line, ring; solid, stroke, corners, gradient
Bitmap	BitmapFactory.decodeResource; scale; OutOfMemoryError risk
Tween animation	alpha, scale, translate, rotate (+ set); res/anim; AnimationUtils.loadAnimation
Frame animation	animation-list + AnimationDrawable; start() onCreate me nahi
Property animation	ObjectAnimator, ValueAnimator, AnimatorSet; API 11

Yaad rakhne ke tricks

Tween animations: A-S-T-R = Alpha, Scale, Translate, Rotate.
Layouts: L-R-F-T-G-C = Linear, Relative, Frame, Table, Grid, Constraint.
gravity = view ke **andar**. **layout_gravity** = view **parent me**.
dp = **d**istance, **sp** = text ka **s**cale.
Animation types: V-F-P = View (tween), Frame, Property.

Common Mistakes jo Students Karte Hain

- **dp** aur **sp** ki jagah **px** use karna. Tab UI har phone par alag dikhta hai.
- **gravity** aur **layout_gravity** ko mix karna.
- Weights use karna par size **0dp** na rakhna.
- Deprecated cheezein (AbsoluteLayout, ProgressDialog, DigitalClock) ko normal tareeka likhna.
- Animation me **duration** bhool jana, ya frame **start()** ko onCreate() me call karna.
- Ye bhool jana ki **date-picker ka month 0 se shuru** hota hai.

Self-Test (bina dekhe likho, har question 5 marks)

- Vertical LinearLayout ki XML likho jisme TextView, EditText aur weights se side by side do buttons hon.
- LinearLayout aur RelativeLayout me difference likho. Baaki layouts ke naam likho.
- Spinner ke selected item ke saath Toast dikhane ka program likho.
- Tween animation explain karo. Fade-in aur rotate animation ki XML likho.
- Canvas aur Paint kya hain? Circle draw karne ka code likho.

Self-Test Answers (likhne ke baad check karo)

1. LinearLayout orientation=vertical jisme TextView, EditText (match_parent width) aur ek inner horizontal LinearLayout jisme do Buttons: dono ke liye layout_width=0dp aur layout_weight=1. Q7 dekho.

2. LinearLayout: ek row ya column, orientation aur weight. RelativeLayout: parent ya siblings ke relative positions. Baaki: FrameLayout, TableLayout, GridLayout, ConstraintLayout.

3. Spinner par ArrayAdapter lagao, phir setOnItemSelectedListener: onItemSelected() me parent.getItemAtPosition(pos).toString() aur Toast.makeText(...).show(). Q4 dekho.

4. Tween animation start aur end values deta hai aur system beech ke frames banata hai. Fade-in: <alpha fromAlpha="0.0" toAlpha="1.0" duration="1000"/>. Rotate: <rotate fromDegrees="0" toDegrees="360" pivotX="50%" pivotY="50%" duration="1000"/>.

5. Canvas drawing surface hai jisme draw methods hote hain; Paint colour, style aur size rakhta hai. Code: Paint p = new Paint(); p.setColor(Color.RED); canvas.drawCircle(200, 200, 100, p); custom View ke onDraw() me.

Unit III complete!
Agar 5 me se 4 ya zyada sahi hue to **Unit IV** par jao. Nahi to Part A me jo topics miss hue unhe dobara padho.